BÁO CÁO BTVN#3

Agent đặt vé máy bay với lớp Harness

Huỳnh An Phú – 24521351

Lớp: SE373.R11

Nộp kèm: flight_agent.py (mã nguồn, chạy bằng python flight_agent.py).

1. Đề bài yêu cầu

Tìm hiểu Langchain, LangGraph → Tạo tool mockup → Viết lớp harness cho Agent này. Nộp .py kèm báo cáo.

1. Cài đặt đủ các lớp harness: ràng buộc là dữ liệu, tiêu chí hoàn thành kiểm bằng code, kiểm quyền, bàn giao;

2. Cài đặt Agent với 3 mẫu thiết kế: ReAct, Plan-then-Execute, Lai

3. Đánh giá hiệu quả của Agent với 3 mẫu thiết kế khác nhau

2. Tìm hiểu LangChain và LangGraph

2.1 LangChain (create_agent và middleware)

Từ LangChain 1.0, create_agent là cách chuẩn để dựng agent; nó thay thế create_react_agent trước đây nằm trong langgraph.prebuilt. Hàm này dựng vòng lặp agent cơ bản: gọi model, để model chọn công cụ cần chạy, rồi kết thúc khi model không gọi công cụ nào nữa.

Tính năng đặc trưng của create_agent là **middleware**: các hook chen vào nhiều điểm trong vòng lặp. Tài liệu LangChain mô tả create_agent là một harness tối thiểu, có thể cấu hình, trong đó harness là mọi thứ bao quanh vòng lặp model: prompt, công cụ và middleware.

2.2 LangGraph

LangGraph là framework điều phối ở tầng thấp hơn. Hàm create_agent được xây trên LangGraph nên thừa hưởng thực thi bền vững , hỗ trợ human-in-the-loop và lưu trạng thái. Dùng LangGraph khi cần kết hợp luồng xác định với luồng agent tự quyết; đó chính là tình huống của mẫu Plan-then-Execute và mẫu Lai.

3. Công cụ giả lập (mockup tools)

Bốn công cụ dùng mock data, lưu các chuyến bay vào cơ sở dữ liệu giả BOOKINGS và FLIGHT. Có hai bộ dữ liệu chuyến bay để kiểm tra hai tình huống: success (có chuyến hợp lệ) và fail (không chuyến nào vừa buổi sáng vừa dưới 2 triệu).

Công cụ	**Chức năng**
search_flights	Tìm chuyến theo tuyến và ngày
book_seat	Giữ chỗ, trả mã đặt chỗ
pay	Thanh toán booking đã giữ chỗ
get_booking	Đọc lại booking từ hệ thống

4. Các lớp harness

Cả sáu lớp dưới đây là mã Python xác định, dùng chung cho cả ba mẫu. Điểm mấu chốt: model không thể "thương lượng" với các lớp này.

Lớp	**Hàm / lớp**	**Chạy khi nào**	**Vấn đề giải quyết**
1. Ràng buộc là dữ liệu	Constraints (dataclass)	Dựng prompt và kiểm tra	Yêu cầu người dùng được lưu thành dữ liệu; prompt sinh ra từ dữ liệu, và is_ok() kiểm cùng dữ liệu đó. Model không thể "quên" hay diễn giải lại.
2. Kiểm quyền	check_permission() + guarded_call()	TRƯỚC mỗi lời gọi công cụ	Chặn book_seat với chuyến vi phạm ràng buộc; công cụ không chạy, model nhận denied kèm lý do.
3. Hoàn thành kiểm bằng code	is_done()	Sau khi agent dừng	Đọc lại booking bằng get_booking: phải tồn tại, đã thanh toán và thỏa ràng buộc. Không tin câu "tôi xong rồi" của model.
4. Bàn giao	handoff()	Khi kết quả là FAILED	Đưa cho người 3 thứ: đã làm gì, đã thử gì, và câu hỏi cần quyết định.
5. Kiểm tra kế hoạch	validate_plan()	Trước khi chạy bước nào của plan	Kế hoạch cũng là dữ liệu: chỉ cho dùng công cụ trong danh

Lớp	Hàm / lớp	Chạy khi nào	Vấn đề giải quyết
			sách, tối đa 6 bước, pay phải đứng sau book_seat.
6. Người dùng duyệt kế hoạch	confirm_plan()	Sau validate_plan, trước thực thi	Plan có bước pay tiêu tiền, nên không bước nào chạy khi chưa có "y". Từ chối thì dừng hẳn.

5. Ba mẫu thiết kế của Agent

Người dùng chọn mẫu khi chạy chương trình. Cả ba dùng cùng công cụ, cùng harness và cùng kiểm tra is_done(); chúng chỉ khác ở cách model ra quyết định.

5.1 ReAct (suy nghĩ, hành động, quan sát)

Vòng lặp do create_agent của LangChain chạy: model chọn một lời gọi công cụ, đọc kết quả, rồi chọn bước tiếp theo, cho tới khi không gọi công cụ nữa. Harness chen vào qua middleware. Khi một bước bị denied, model nhìn thấy lý do ngay trong lượt kế tiếp nên có thể tự đổi hướng. Đổi lại, mỗi bước là một lần gọi model, và hành vi khó đoán trước hơn.

5.2 Plan-then-Execute (lập kế hoạch rồi thực thi)

1. **Plan:** planner viết cả kế hoạch một lần (JSON gồm các bước tool, goal, args).
2. **Validate:** validate_plan() kiểm kế hoạch bằng code.
3. **Approve:** confirm_plan() hỏi người dùng; trả lời "n" thì dừng, không công cụ nào chạy.
4. **Execute:** với mỗi bước, worker điền tham số (dùng kết quả các bước trước), rồi guarded_call() chạy và không lập kế hoạch lại.

5.3 Lai (Plan-then-Execute, ReAct)

Chạy Plan-then-Execute trước . Nếu kế hoạch hỏng, hoặc chạy xong mà is_done() vẫn sai, hệ thống chuyển sang ReAct dự phòng: agent ReAct nhận prompt gồm ràng buộc và toàn bộ lịch sử những gì kế hoạch đã làm, rồi tiếp tục. Harness vẫn chặn mọi lời gọi sai. Hai quy tắc quan trọng:

- Nếu người dùng từ chối kế hoạch thì dừng hẳn, không được chuyển sang ReAct để lách quyết định của người dùng.
- ReAct dự phòng bị giới hạn ngân sách 10 lần gọi model như ReAct thường.

6. Đánh giá hiệu quả của ba mẫu

6.1 Thiết lập thực nghiệm

Chạy 3 mẫu × 2 tình huống (success, fail) = 6 lượt, người dùng duyệt kế hoạch ở các mẫu có kế hoạch; thêm 2 lượt khi người dùng từ chối kế hoạch (Plan-then-Execute và Lai, tình huống success). Số liệu lấy từ LOG và BOOKINGS của chương trình. Số lần gọi model được tính theo thiết kế (mỗi lời gọi công cụ hoặc câu trả lời cuối của ReAct là một lần; Plan có 1 lần lập kế hoạch và 1 lần cho mỗi bước cần điền tham số), vì model là giả lập.

6.2 Kết quả

Mẫu	Tình huống	Kết quả	Gọi công cụ	Bị chặn	Đã trả tiền	Gọi model	Người dùng duyệt
ReAct	success	DONE	3	0	1	4	0
ReAct	fail	FAILED + bàn giao	2	1	0	3	0
Plan-then-Execute	success	DONE	3	0	1	3	1
Plan-then-Execute	fail	FAILED + bàn giao	2	1	0	2	1
Lai	success	DONE	3	0	1	3	1
Lai	fail	FAILED + bàn giao	2	1	0	3	1

Khi người dùng từ chối kế hoạch (tình huống success):

Mẫu	Người dùng trả lời	Gọi công cụ	Kết quả	Chuyển sang ReAct?
Plan-then-Execute	n (từ chối)	0	FAILED, câu hỏi: “bạn muốn sửa gì trong kế hoạch?”	Không áp dụng
Lai	n (từ chối)	0	FAILED, câu hỏi như trên	Không

6.3 So sánh giữa 3 mẫu suy luận

Tiêu chí	ReAct	Plan-then-Execute	Lai
Khả năng thích nghi khi bước sai	Cao: thấy lý do bị chặn và đổi hướng	Thấp: dừng, không lập kế hoạch lại	Cao: ReAct dự phòng
Dự đoán được hành vi	Thấp	Cao: thấy trước toàn bộ kế hoạch	Cao ở pha đầu, thấp hơn ở pha dự phòng
Chi phí gọi model	Tăng theo số bước	Thấp (1 lần lập kế hoạch + bước)	Như Plan, cộng ReAct khi cần
Xác nhận của người dùng	Không có sẵn	Có, trước khi chạy	Có, trước khi chạy
Độ phức tạp cài đặt	Thấp	Trung bình	Cao nhất (hai cơ chế)

7. Hướng dẫn chạy

```
pip install -U langchain
python flight_agent.py
# 1) chọn mẫu: ReAct / Plan-then-Execute / Lai
# 2) chọn tình huống: success / fail
# 3) với Plan-then-Execute và Lai: trả lời y/n khi được hỏi duyệt kế hoạch
```

Chương trình in lần lượt: kế hoạch (nếu có), vết gọi công cụ (trace, kèm nhãn giai đoạn react / plan / fallback) và kết quả cuối cùng DONE hoặc FAILED kèm bàn giao.